

DATA STRUCTURES PLACEMENT PREPARATION GUIDE

Complexity Analysis • Arrays • Searching & Sorting
Strings • Linked Lists (Singly / Doubly / Circular)
Stacks – ADT, Implementations & Applications

Concept Explanations + Two Java Programs for Every Topic

Table of Contents

1. Time Complexity and Big-O Notation
2. Space Complexity and Auxiliary Space
3. One-Dimensional Arrays
4. Two-Dimensional and Multidimensional Arrays
5. Array Operations: Traversal, Insertion, Deletion, Searching
6. Searching: Linear and Binary Search
7. Sorting Arrays and Array-Based Problems
8. String Representation and Basic Operations
9. String Searching and Pattern Matching
10. String Manipulation and Common Interview Problems
11. Singly Linked List
12. Doubly Linked List
13. Circular Linked List
14. Linked List Operations and Applications
15. Reverse, Merge, Detect Cycle and Remove Duplicates
16. Stack ADT and Basic Operations
17. Stack Implementation Using Array and Linked List
18. Stack Applications: Expression Conversion/Evaluation, Parentheses, Call Stack

1. Time Complexity and Big-O Notation

Time complexity measures how the running time of an algorithm grows with input size **n**. Big-O notation describes the **upper bound** (worst-case growth rate), ignoring constants and lower-order terms.

Common Complexities (best → worst)

$O(1)$ constant $O(\log n)$ logarithmic $O(n)$ linear $O(n \log n)$ linearithmic $O(n^2)$ quadratic $O(n^3)$ cubic $O(2^n)$ exponential $O(n!)$ factorial

Rules of thumb: loops → multiply by n ; nested loops → n^2 ; divide-and-conquer (binary search, merge sort) → $\log n$; recursive tree with branching → often exponential unless memoized.

Example 1: Identifying complexity of simple loops

```
public class TimeComplexity1 {
    // O(1)
    static int getFirst(int[] a) { return a[0]; }

    // O(n)
    static int sum(int[] a) {
        int s = 0;
        for (int x : a) s += x;
        return s;
    }

    // O(n²)
    static void printPairs(int[] a) {
        for (int i = 0; i < a.length; i++)
            for (int j = 0; j < a.length; j++)
                System.out.println(a[i] + "," + a[j]);
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4};
        System.out.println(getFirst(arr));
        System.out.println(sum(arr));
    }
}
```

Example 2: Binary search style – $O(\log n)$

```
public class TimeComplexity2 {
    // O(log n) – each step halves the search space
    static int powerOfTwoSteps(int n) {
        int steps = 0;
        while (n > 1) {
            n = n / 2;
            steps++;
        }
        return steps;
    }

    public static void main(String[] args) {
        System.out.println(powerOfTwoSteps(16)); // 4 (16→8→4→2→1)
        System.out.println(powerOfTwoSteps(1024)); // 10
    }
}
```

2. Space Complexity and Auxiliary Space

Space complexity = total memory used relative to input size n (including input itself). **Auxiliary space** = extra space used by the algorithm *excluding* the input. Interviewers often care about auxiliary space.

Examples: in-place algorithms $\rightarrow O(1)$ auxiliary; creating a new array of size $n \rightarrow O(n)$; recursion depth $d \rightarrow O(d)$ stack space.

Example 1: $O(1)$ auxiliary vs $O(n)$ auxiliary

```
public class Space1 {
    // O(1) auxiliary - modify in place
    static void reverseInPlace(int[] a) {
        int i = 0, j = a.length - 1;
        while (i < j) {
            int t = a[i]; a[i] = a[j]; a[j] = t;
            i++; j--;
        }
    }
    // O(n) auxiliary - new array
    static int[] reverseNew(int[] a) {
        int[] b = new int[a.length];
        for (int i = 0; i < a.length; i++)
            b[i] = a[a.length - 1 - i];
        return b;
    }
    public static void main(String[] args) {
        int[] a = {1, 2, 3, 4};
        reverseInPlace(a);
        for (int x : a) System.out.print(x + " "); // 4 3 2 1
    }
}
```

Example 2: Recursion stack space

```
public class Space2 {
    // Auxiliary space O(n) due to call stack
    static int factorial(int n) {
        if (n <= 1) return 1;
        return n * factorial(n - 1);
    }
    // Iterative - O(1) auxiliary
    static int factorialIter(int n) {
        int res = 1;
        for (int i = 2; i <= n; i++) res *= i;
        return res;
    }
    public static void main(String[] args) {
        System.out.println(factorial(5));
        System.out.println(factorialIter(5));
    }
}
```

3. One-Dimensional Arrays

Contiguous memory, fixed size, $O(1)$ random access by index. Creation: `int[] a = new int[n];` or initializer list. Default values: 0 / 0.0 / false / null. Index range 0 ... length-1.

Example 1: Creation, access, bounds

```
public class OneDArray1 {
    public static void main(String[] args) {
        int[] a = new int[5];           // {0,0,0,0,0}
        int[] b = {10, 20, 30, 40, 50};
        System.out.println(b[2]);       // 30
        System.out.println(b.length);  // 5
        // b[5] → ArrayIndexOutOfBoundsException
        for (int i = 0; i < b.length; i++)
            System.out.print(b[i] + " ");
    }
}
```

Example 2: Prefix sum (common array technique)

```
public class OneDArray2 {
    static int[] prefixSum(int[] a) {
        int[] pre = new int[a.length];
        pre[0] = a[0];
        for (int i = 1; i < a.length; i++)
            pre[i] = pre[i - 1] + a[i];
        return pre;
    }
    public static void main(String[] args) {
        int[] a = {2, 4, 6, 8};
        int[] p = prefixSum(a); // 2, 6, 12, 20
        for (int x : p) System.out.print(x + " ");
        // range sum [1..3] = p[3] - p[0] = 18
    }
}
```

4. Two-Dimensional and Multidimensional Arrays

Array of arrays. `int[][] m = new int[r][c]`; Jagged arrays allowed (rows of different lengths). Access: `m[i][j]`. Common uses: matrices, grids, DP tables.

Example 1: Matrix creation & print

```
public class TwoD1 {
    public static void main(String[] args) {
        int[][] m = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
        for (int i = 0; i < m.length; i++) {
            for (int j = 0; j < m[i].length; j++)
                System.out.print(m[i][j] + " ");
            System.out.println();
        }
    }
}
```

Example 2: Diagonal sum & jagged array

```
public class TwoD2 {
    public static void main(String[] args) {
        int[][] m = {{1,2,3},{4,5,6},{7,8,9}};
        int primary = 0, secondary = 0, n = m.length;
        for (int i = 0; i < n; i++) {
            primary += m[i][i];
            secondary += m[i][n - 1 - i];
        }
        System.out.println("Primary diagonal: " + primary);
        System.out.println("Secondary diagonal: " + secondary);

        // Jagged
        int[][] jag = new int[3][];
        jag[0] = new int[]{1};
        jag[1] = new int[]{2, 3};
        jag[2] = new int[]{4, 5, 6};
    }
}
```

5. Array Operations: Traversal, Insertion, Deletion, Searching

Because size is fixed, insert/delete need shifting ($O(n)$). Traversal $O(n)$. Search covered next. Use a logical size when the physical array is larger.

Example 1: Insert & delete with logical size

```
public class ArrayOps1 {
    static int insert(int[] a, int n, int pos, int val) {
        if (n == a.length || pos < 0 || pos > n) return n;
        for (int i = n; i > pos; i--) a[i] = a[i - 1];
        a[pos] = val;
        return n + 1;
    }
    static int deleteAt(int[] a, int n, int pos) {
        if (pos < 0 || pos >= n) return n;
        for (int i = pos; i < n - 1; i++) a[i] = a[i + 1];
        return n - 1;
    }
    public static void main(String[] args) {
        int[] a = new int[8];
        a[0]=10; a[1]=20; a[2]=30; a[3]=40;
        int n = 4;
        n = insert(a, n, 2, 25);
        n = deleteAt(a, n, 1);
        for (int i = 0; i < n; i++) System.out.print(a[i] + " ");
        // 10 25 30 40
    }
}
```

Example 2: Traversal techniques

```
public class ArrayOps2 {
    public static void main(String[] args) {
        int[] a = {5, 10, 15, 20};
        // forward
        for (int i = 0; i < a.length; i++) System.out.print(a[i] + " ");
        System.out.println();
        // reverse
        for (int i = a.length - 1; i >= 0; i--) System.out.print(a[i] + " ");
        System.out.println();
        // for-each
        for (int x : a) System.out.print(x + " ");
    }
}
```

6. Searching in Arrays: Linear and Binary Search

Linear Search – $O(n)$, works on unsorted data.

Binary Search – $O(\log n)$, requires sorted array. Repeatedly halves the search space.

Example 1: Linear Search

```
public class LinearSearch {
    static int linear(int[] a, int key) {
        for (int i = 0; i < a.length; i++)
            if (a[i] == key) return i;
        return -1;
    }
    public static void main(String[] args) {
        int[] a = {4, 2, 7, 1, 9};
        System.out.println(linear(a, 7)); // 2
        System.out.println(linear(a, 5)); // -1
    }
}
```

Example 2: Binary Search (iterative)

```
public class BinarySearch {
    static int binary(int[] a, int key) {
        int lo = 0, hi = a.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] == key) return mid;
            if (a[mid] < key) lo = mid + 1;
            else hi = mid - 1;
        }
        return -1;
    }
    public static void main(String[] args) {
        int[] a = {1, 3, 5, 7, 9, 11};
        System.out.println(binary(a, 7)); // 3
        System.out.println(binary(a, 4)); // -1
    }
}
```

7. Sorting Arrays and Array-Based Problems

Common simple sorts: Bubble $O(n^2)$, Selection $O(n^2)$, Insertion $O(n^2)$. Efficient: Merge Sort $O(n \log n)$, Quick Sort average $O(n \log n)$, Heap Sort $O(n \log n)$. Java: `Arrays.sort()` (Dual-Pivot Quicksort for primitives, TimSort for objects).

Example 1: Bubble Sort

```
public class BubbleSort {
    static void bubble(int[] a) {
        int n = a.length;
        for (int i = 0; i < n - 1; i++)
            for (int j = 0; j < n - 1 - i; j++)
                if (a[j] > a[j + 1]) {
                    int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                }
    }
    public static void main(String[] args) {
        int[] a = {5, 1, 4, 2, 8};
        bubble(a);
        for (int x : a) System.out.print(x + " "); // 1 2 4 5 8
    }
}
```

Example 2: Arrays.sort + two-sum style problem

```
import java.util.Arrays;
public class SortProblems {
    // Check if any two elements sum to target (after sort + two pointers)
    static boolean twoSum(int[] a, int target) {
        Arrays.sort(a);
        int i = 0, j = a.length - 1;
        while (i < j) {
            int s = a[i] + a[j];
            if (s == target) return true;
            if (s < target) i++;
            else j--;
        }
        return false;
    }
    public static void main(String[] args) {
        System.out.println(twoSum(new int[]{3, 1, 8, 4}, 7)); // true (3+4)
    }
}
```

8. String Representation and Basic Operations

In Java, String is immutable, backed by a char array (or byte array in newer JDKs). Basic ops: length, charAt, substring, concat (+), comparison (equals / compareTo).

Example 1: Basic operations

```
public class StringBasic1 {
    public static void main(String[] args) {
        String s = "Hello";
        System.out.println(s.length());
        System.out.println(s.charAt(1));
        System.out.println(s.substring(1, 4));
        System.out.println(s + " World");
        System.out.println(s.equals("Hello"));
        System.out.println("abc".compareTo("abd")); // negative
    }
}
```

Example 2: Convert to char array & back

```
public class StringBasic2 {
    public static void main(String[] args) {
        String s = "Java";
        char[] ch = s.toCharArray();
        ch[0] = 'j';
        String t = new String(ch);
        System.out.println(t);           // java
        System.out.println(s);           // Java (immutable)
    }
}
```

9. String Searching and Pattern Matching

Naive pattern search $O((n-m+1)*m)$. Built-in: `indexOf`, `contains`, `matches` (regex). Classic algorithms: KMP, Rabin-Karp, Boyer-Moore (interview awareness).

Example 1: Naive pattern search

```
public class PatternSearch1 {
    static int naiveSearch(String text, String pat) {
        int n = text.length(), m = pat.length();
        for (int i = 0; i <= n - m; i++) {
            int j;
            for (j = 0; j < m; j++)
                if (text.charAt(i + j) != pat.charAt(j)) break;
            if (j == m) return i;
        }
        return -1;
    }
    public static void main(String[] args) {
        System.out.println(naiveSearch("ababcababababd", "ababd")); // 10
        System.out.println("hello".indexOf("ll")); // 2
    }
}
```

Example 2: Count occurrences of a pattern

```
public class PatternSearch2 {
    static int countOccurrences(String text, String pat) {
        int count = 0, idx = 0;
        while ((idx = text.indexOf(pat, idx)) != -1) {
            count++;
            idx += pat.length(); // non-overlapping; use idx++ for overlapping
        }
        return count;
    }
    public static void main(String[] args) {
        System.out.println(countOccurrences("abababa", "aba")); // 2
    }
}
```

10. String Manipulation and Common Interview Problems

Frequent problems: reverse, palindrome, anagram, first unique character, longest substring without repeating chars, run-length encoding, etc. Prefer StringBuilder for heavy modification.

Example 1: Palindrome & Anagram check

```
import java.util.Arrays;
public class StringProblems1 {
    static boolean isPalindrome(String s) {
        int i = 0, j = s.length() - 1;
        while (i < j) if (s.charAt(i++) != s.charAt(j--)) return false;
        return true;
    }
    static boolean isAnagram(String a, String b) {
        if (a.length() != b.length()) return false;
        char[] x = a.toCharArray(), y = b.toCharArray();
        Arrays.sort(x); Arrays.sort(y);
        return Arrays.equals(x, y);
    }
    public static void main(String[] args) {
        System.out.println(isPalindrome("racecar")); // true
        System.out.println(isAnagram("listen", "silent")); // true
    }
}
```

Example 2: First non-repeating character

```
public class StringProblems2 {
    static Character firstUnique(String s) {
        int[] freq = new int[256];
        for (char c : s.toCharArray()) freq[c]++;
        for (char c : s.toCharArray())
            if (freq[c] == 1) return c;
        return null;
    }
    public static void main(String[] args) {
        System.out.println(firstUnique("swiss")); // w
    }
}
```

11. Singly Linked List

Each node has **data** + **next** reference. Dynamic size, O(1) insert/delete at head (if head reference available), O(n) access by index. No random access.

Example 1: Node class + insert at head & traverse

```
class Node {
    int data;
    Node next;
    Node(int d) { data = d; }
}
public class SinglyLL1 {
    static Node head;
    static void insertHead(int val) {
        Node n = new Node(val);
        n.next = head;
        head = n;
    }
    static void print() {
        Node cur = head;
        while (cur != null) {
            System.out.print(cur.data + " -> ");
            cur = cur.next;
        }
        System.out.println("null");
    }
    public static void main(String[] args) {
        insertHead(30); insertHead(20); insertHead(10);
        print(); // 10 -> 20 -> 30 -> null
    }
}
```

Example 2: Insert at end & length

```
public class SinglyLL2 {
    static Node head;
    static void insertEnd(int val) {
        Node n = new Node(val);
        if (head == null) { head = n; return; }
        Node cur = head;
        while (cur.next != null) cur = cur.next;
        cur.next = n;
    }
    static int length() {
        int c = 0; Node cur = head;
        while (cur != null) { c++; cur = cur.next; }
        return c;
    }
    public static void main(String[] args) {
        insertEnd(1); insertEnd(2); insertEnd(3);
        System.out.println("Length = " + length()); // 3
    }
}
```

12. Doubly Linked List

Each node has **prev** + **data** + **next**. Can traverse both directions. Insert/delete need extra pointer updates but allow $O(1)$ delete when node reference is known.

Example 1: Doubly node + insert at head

```
class DNode {
    int data;
    DNode prev, next;
    DNode(int d) { data = d; }
}
public class DoublyLL1 {
    static DNode head;
    static void insertHead(int val) {
        DNode n = new DNode(val);
        n.next = head;
        if (head != null) head.prev = n;
        head = n;
    }
    static void printForward() {
        DNode cur = head;
        while (cur != null) {
            System.out.print(cur.data + " ");
            cur = cur.next;
        }
        System.out.println();
    }
    public static void main(String[] args) {
        insertHead(3); insertHead(2); insertHead(1);
        printForward(); // 1 2 3
    }
}
```

Example 2: Traverse backward from tail

```
public class DoublyLL2 {
    static DNode head;
    static void insertHead(int val) {
        DNode n = new DNode(val);
        n.next = head;
        if (head != null) head.prev = n;
        head = n;
    }
    static void printBackward() {
        if (head == null) return;
        DNode cur = head;
        while (cur.next != null) cur = cur.next; // go to tail
        while (cur != null) {
            System.out.print(cur.data + " ");
            cur = cur.prev;
        }
        System.out.println();
    }
    public static void main(String[] args) {
        insertHead(3); insertHead(2); insertHead(1);
        printBackward(); // 3 2 1
    }
}
```

13. Circular Linked List

Last node's next points back to head (singly circular) or both ends connect (doubly circular). Useful for round-robin, Josephus problem, continuous playlists. Careful with termination conditions.

Example 1: Singly circular – insert & traverse once

```
public class CircularLL1 {
    static Node head;
    static void insertEnd(int val) {
        Node n = new Node(val);
        if (head == null) {
            head = n; n.next = head; return;
        }
        Node cur = head;
        while (cur.next != head) cur = cur.next;
        cur.next = n;
        n.next = head;
    }
    static void printOnce() {
        if (head == null) return;
        Node cur = head;
        do {
            System.out.print(cur.data + " ");
            cur = cur.next;
        } while (cur != head);
        System.out.println();
    }
    public static void main(String[] args) {
        insertEnd(10); insertEnd(20); insertEnd(30);
        printOnce(); // 10 20 30
    }
}
```

Example 2: Detect if a list is circular (Floyd idea preview)

```
public class CircularLL2 {
    static boolean isCircular(Node head) {
        if (head == null) return false;
        Node slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }
    public static void main(String[] args) {
        Node a = new Node(1), b = new Node(2), c = new Node(3);
        a.next = b; b.next = c; c.next = a; // circular
        System.out.println(isCircular(a)); // true
    }
}
```

14. Linked List Operations and Applications

Core ops: insert (head/tail/middle), delete (by value/position), search, length, find middle (slow-fast), nth from end. Applications: implement Stack/Queue, adjacency lists, undo history, polynomial arithmetic, memory management freelists.

Example 1: Find middle (tortoise-hare)

```
public class LLOps1 {
    static Node middle(Node head) {
        Node slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }
    public static void main(String[] args) {
        Node h = new Node(1);
        h.next = new Node(2); h.next.next = new Node(3);
        h.next.next.next = new Node(4); h.next.next.next.next = new Node(5);
        System.out.println(middle(h).data); // 3
    }
}
```

Example 2: nth node from end (two pointers)

```
public class LLOps2 {
    static Node nthFromEnd(Node head, int n) {
        Node fast = head, slow = head;
        for (int i = 0; i < n; i++) {
            if (fast == null) return null;
            fast = fast.next;
        }
        while (fast != null) {
            slow = slow.next;
            fast = fast.next;
        }
        return slow;
    }
    public static void main(String[] args) {
        Node h = new Node(10); h.next = new Node(20);
        h.next.next = new Node(30); h.next.next.next = new Node(40);
        System.out.println(nthFromEnd(h, 2).data); // 30
    }
}
```

15. Reverse, Merge, Detect Cycle and Remove Duplicates

Must-know interview problems on linked lists. Reverse: iterative with 3 pointers or recursive. Cycle: Floyd's cycle-finding (tortoise & hare). Merge two sorted lists. Remove duplicates from sorted/unordered list.

Example 1: Reverse iterative + Detect cycle

```
public class LLAdvanced1 {
    static Node reverse(Node head) {
        Node prev = null, cur = head;
        while (cur != null) {
            Node next = cur.next;
            cur.next = prev;
            prev = cur;
            cur = next;
        }
        return prev;
    }
    static boolean hasCycle(Node head) {
        Node slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }
    public static void main(String[] args) {
        Node h = new Node(1); h.next = new Node(2); h.next.next = new Node(3);
        h = reverse(h);
        System.out.println(h.data); // 3
    }
}
```

Example 2: Merge two sorted lists + remove duplicates (sorted)

```
public class LLAdvanced2 {
    static Node merge(Node a, Node b) {
        Node dummy = new Node(0), tail = dummy;
        while (a != null && b != null) {
            if (a.data <= b.data) { tail.next = a; a = a.next; }
            else { tail.next = b; b = b.next; }
            tail = tail.next;
        }
        tail.next = (a != null) ? a : b;
        return dummy.next;
    }
    static Node removeDuplicates(Node head) {
        Node cur = head;
        while (cur != null && cur.next != null) {
            if (cur.data == cur.next.data) cur.next = cur.next.next;
            else cur = cur.next;
        }
        return head;
    }
}
```

16. Stack ADT and Basic Operations

Stack = LIFO (Last-In-First-Out). Abstract Data Type operations:

- **push(x)** – insert at top
- **pop()** – remove & return top
- **peek/top()** – return top without removal
- **isEmpty()** / **size()**

All of the above are $O(1)$ in a proper implementation.

Example 1: Conceptual usage with java.util.Stack

```
import java.util.Stack;
public class StackADT1 {
    public static void main(String[] args) {
        Stack<Integer> st = new Stack<>();
        st.push(10); st.push(20); st.push(30);
        System.out.println(st.peek()); // 30
        System.out.println(st.pop());  // 30
        System.out.println(st.pop());  // 20
        System.out.println(st.isEmpty()); // false
        System.out.println(st.size());  // 1
    }
}
```

Example 2: Underflow / Overflow awareness

```
public class StackADT2 {
    public static void main(String[] args) {
        Stack<String> st = new Stack<>();
        // st.pop(); → EmptyStackException (underflow)
        st.push("A"); st.push("B");
        while (!st.isEmpty())
            System.out.print(st.pop() + " "); // B A
    }
}
```

17. Stack Implementation Using Array and Linked List

Array-based: fixed or dynamic capacity, top index. Fast, cache-friendly; overflow possible if fixed. Linked-list-based: push/pop at head, no overflow (until memory), extra pointer overhead.

Example 1: Array-based Stack

```
class ArrayStack {
    private int[] a;
    private int top;
    ArrayStack(int cap) { a = new int[cap]; top = -1; }
    void push(int x) {
        if (top == a.length - 1) throw new RuntimeException("Overflow");
        a[++top] = x;
    }
    int pop() {
        if (top == -1) throw new RuntimeException("Underflow");
        return a[top--];
    }
    int peek() { return a[top]; }
    boolean isEmpty() { return top == -1; }
}

public class StackImpl1 {
    public static void main(String[] args) {
        ArrayStack st = new ArrayStack(5);
        st.push(1); st.push(2); st.push(3);
        System.out.println(st.pop()); // 3
        System.out.println(st.peek()); // 2
    }
}
```

Example 2: Linked-list-based Stack

```
class LLStack {
    private Node top;
    void push(int x) {
        Node n = new Node(x);
        n.next = top;
        top = n;
    }
    int pop() {
        if (top == null) throw new RuntimeException("Underflow");
        int v = top.data;
        top = top.next;
        return v;
    }
    boolean isEmpty() { return top == null; }
}

public class StackImpl2 {
    public static void main(String[] args) {
        LLStack st = new LLStack();
        st.push(5); st.push(10);
        System.out.println(st.pop()); // 10
        System.out.println(st.pop()); // 5
    }
}
```

18. Stack Applications: Expression Conversion/Evaluation, Parentheses, Call Stack

Classic applications:

- Balanced parentheses / brackets
- Infix → Postfix (Shunting-yard) and Postfix evaluation
- Function call stack (runtime)
- Undo mechanisms, browser history, DFS backtracking

Example 1: Balanced parentheses

```
import java.util.Stack;
public class StackApp1 {
    static boolean isBalanced(String s) {
        Stack<Character> st = new Stack<>();
        for (char c : s.toCharArray()) {
            if (c == '(' || c == '{' || c == '[') st.push(c);
            else {
                if (st.isEmpty()) return false;
                char t = st.pop();
                if ((c == ')' && t != '(') ||
                    (c == '}' && t != '{') ||
                    (c == ']' && t != '[')) return false;
            }
        }
        return st.isEmpty();
    }
    public static void main(String[] args) {
        System.out.println(isBalanced("[{}]()")); // true
        System.out.println(isBalanced("[{}]()")); // false
    }
}
```

Example 2: Evaluate postfix expression

```
import java.util.Stack;
public class StackApp2 {
    static int evalPostfix(String expr) {
        Stack<Integer> st = new Stack<>();
        for (String token : expr.split(" ")) {
            if (token.matches("-?\\d+")) st.push(Integer.parseInt(token));
            else {
                int b = st.pop(), a = st.pop();
                switch (token) {
                    case "+" -> st.push(a + b);
                    case "-" -> st.push(a - b);
                    case "*" -> st.push(a * b);
                    case "/" -> st.push(a / b);
                }
            }
        }
        return st.pop();
    }
    public static void main(String[] args) {
        // 2 3 * 5 4 * + = 2*3 + 5*4 = 26
        System.out.println(evalPostfix("2 3 * 5 4 * +"));
    }
}
```

You now have full coverage of the listed DSA topics with explanations and two working Java programs each. Practice implementing from scratch and analysing complexity on paper.

[All the best for your placements!](#)